T3000 API & Protocol

Events · built Nov 13, 2026

Events

Event system for alarms, state changes, and notifications.

Event Types

Alarm Events

- Alarm activated
- Alarm acknowledged
- Alarm cleared

Device Events

- Device connected
- Device disconnected
- Communication error

Data Events

- Value changed
- Limit exceeded
- Override activated

Event Structure

JSON

```
{
  "id": "evt_001",
  "type": "alarm",
  "severity": "high",
  "message": "High temperature",
  "timestamp": 1704582000,
  "source": "Zone_Temp",
  "value": 85.2
}
```

Subscribing to Events

Use WebSocket API to receive real-time events.

Event System Architecture

Event Bus Implementation

Core Event Types

TYPESCRIPT

```typescript
interface BaseEvent {
  id: string;
  type: string;
  timestamp: number;
  source: string;
  metadata?: Record<string, any>;
}

interface AlarmEvent extends BaseEvent {
  type: 'alarm' | 'alarm-acknowledged' | 'alarm-cleared';
  alarmId: string;
  deviceId: number;
  point: string;
  severity: 'critical' | 'high' | 'medium' | 'low';
  message: string;
  value?: number;
  threshold?: number;
}

interface DeviceEvent extends BaseEvent {
  type: 'device-connected' | 'device-disconnected' | 'device-error';
  deviceId: number;
  deviceName: string;
  status: 'online' | 'offline' | 'error';
  errorMessage?: string;
}

interface DataEvent extends BaseEvent {
  type: 'value-changed' | 'limit-exceeded' | 'override-activated';
  deviceId: number;
  point: string;
  value: number;
  previousValue?: number;
  limit?: number;
```

```
}

type T3000Event = AlarmEvent | DeviceEvent | DataEvent;
```

Event Emitter

TYPESCRIPT

```typescript
class EventEmitter {
  private listeners = new Map<string, Set<EventListener>>();
  private eventHistory: T3000Event[] = [];
  private maxHistorySize = 1000;

  on(eventType: string, listener: EventListener): () => void {
    if (!this.listeners.has(eventType)) {
      this.listeners.set(eventType, new Set());
    }

    this.listeners.get(eventType)!.add(listener);

    // Return unsubscribe function
    return () => {
      this.listeners.get(eventType)?.delete(listener);
    };
  }

  emit(event: T3000Event): void {
    // Add to history
    this.eventHistory.push(event);
    if (this.eventHistory.length > this.maxHistorySize) {
      this.eventHistory.shift();
    }

    // Notify type-specific listeners
    const typeListeners = this.listeners.get(event.type) || new Set();
    for (const listener of typeListeners) {
      this.safeCall(listener, event);
    }

    // Notify wildcard listeners
    const wildcardListeners = this.listeners.get('*') || new Set();
    for (const listener of wildcardListeners) {
      this.safeCall(listener, event);
    }
  }
```

```
  private safeCall(listener: EventListener, event: T3000Event): void {
    try {
      listener(event);
    } catch (error) {
      console.error('Event listener error:', error);
    }
  }

  getHistory(filter?: (event: T3000Event) => boolean): T3000Event[] {
    return filter
      ? this.eventHistory.filter(filter)
      : [...this.eventHistory];
  }
}

const eventBus = new EventEmitter();
export default eventBus;
```

Event Persistence

Database Schema

SQL

```
CREATE TABLE events (
  id TEXT PRIMARY KEY,
  type TEXT NOT NULL,
  timestamp INTEGER NOT NULL,
  source TEXT NOT NULL,
  device_id INTEGER,
  point TEXT,
  severity TEXT,
  message TEXT,
  data JSON NOT NULL,
  metadata JSON,
  INDEX idx_type_timestamp (type, timestamp),
  INDEX idx_device_timestamp (device_id, timestamp),
  INDEX idx_severity (severity)
);
```

Event Logger

TYPESCRIPT

```typescript
import { Database } from 'sqlite3';

class EventLogger {
  private db: Database;

  constructor(dbPath: string) {
    this.db = new Database(dbPath);
    this.setupTables();

    // Subscribe to all events
    eventBus.on('*', (event) => this.logEvent(event));
  }

  private async logEvent(event: T3000Event): Promise<void> {
    const sql = `
      INSERT INTO events (id, type, timestamp, source, device_id, point, severity, messa
      VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
    `;

    const deviceId = 'deviceId' in event ? event.deviceId : null;
    const point = 'point' in event ? event.point : null;
    const severity = 'severity' in event ? event.severity : null;
    const message = 'message' in event ? event.message : null;

    await this.db.run(sql, [
      event.id,
      event.type,
      event.timestamp,
      event.source,
      deviceId,
      point,
      severity,
      message,
      JSON.stringify(event),
      JSON.stringify(event.metadata || {})
    ]);
  }
```

```
  async queryEvents(options: EventQueryOptions): Promise<T3000Event[]> {
    let sql = 'SELECT data FROM events WHERE 1=1';
    const params: any[] = [];

    if (options.type) {
      sql += ' AND type = ?';
      params.push(options.type);
    }

    if (options.deviceId) {
      sql += ' AND device_id = ?';
      params.push(options.deviceId);
    }

    if (options.severity) {
      sql += ' AND severity = ?';
      params.push(options.severity);
    }

    if (options.startTime) {
      sql += ' AND timestamp >= ?';
      params.push(options.startTime);
    }

    if (options.endTime) {
      sql += ' AND timestamp <= ?';
      params.push(options.endTime);
    }

    sql += ' ORDER BY timestamp DESC LIMIT ?';
    params.push(options.limit || 100);

    const rows = await this.db.all(sql, params);
    return rows.map(row => JSON.parse(row.data));
  }
}
```

Event Filtering and Aggregation

Event Filters

TYPESCRIPT

```typescript
class EventFilter {
  static byDevice(deviceId: number) {
    return (event: T3000Event) => {
      return 'deviceId' in event && event.deviceId === deviceId;
    };
  }

  static byPoint(deviceId: number, point: string) {
    return (event: T3000Event) => {
      return 'deviceId' in event &&
             'point' in event &&
             event.deviceId === deviceId &&
             event.point === point;
    };
  }

  static bySeverity(severity: string) {
    return (event: T3000Event) => {
      return 'severity' in event && event.severity === severity;
    };
  }

  static byTimeRange(start: number, end: number) {
    return (event: T3000Event) => {
      return event.timestamp >= start && event.timestamp <= end;
    };
  }
}

// Usage
eventBus.on('alarm', (event) => {
  if (EventFilter.bySeverity('critical')(event)) {
    sendEmailNotification(event);
  }
});
```

Change-of-Value Detection

TYPESCRIPT

```typescript
class COVDetector {
  private lastValues = new Map<string, number>();
  private thresholds = new Map<string, number>();

  setThreshold(point: string, threshold: number): void {
    this.thresholds.set(point, threshold);
  }

  check(deviceId: number, point: string, value: number): void {
    const key = `${deviceId}:${point}`;
    const lastValue = this.lastValues.get(key);
    const threshold = this.thresholds.get(point) || 0.1;

    if (lastValue !== undefined) {
      const change = Math.abs(value - lastValue);

      if (change >= threshold) {
        eventBus.emit({
          id: generateEventId(),
          type: 'value-changed',
          timestamp: Date.now(),
          source: 'cov-detector',
          deviceId,
          point,
          value,
          previousValue: lastValue
        });
      }
    }

    this.lastValues.set(key, value);
  }
}
```

WebSocket Event Streaming

TYPESCRIPT

```typescript
import WebSocket from 'ws';

class EventWebSocketHandler {
  private subscriptions = new Map<WebSocket, EventSubscription>();

  constructor() {
    eventBus.on('*', (event) => this.broadcastEvent(event));
  }

  handleConnection(ws: WebSocket): void {
    this.subscriptions.set(ws, {
      eventTypes: new Set(['*']),
      deviceIds: new Set(),
      severity: null
    });

    ws.on('message', (data) => {
      const message = JSON.parse(data.toString());
      this.handleMessage(ws, message);
    });

    ws.on('close', () => {
      this.subscriptions.delete(ws);
    });
  }

  private handleMessage(ws: WebSocket, message: any): void {
    if (message.type === 'subscribe') {
      const sub = this.subscriptions.get(ws)!;

      if (message.eventTypes) {
        sub.eventTypes = new Set(message.eventTypes);
      }

      if (message.deviceIds) {
        sub.deviceIds = new Set(message.deviceIds);
      }
```

```
      if (message.severity) {
        sub.severity = message.severity;
      }
    }
  }

  private broadcastEvent(event: T3000Event): void {
    for (const [ws, sub] of this.subscriptions.entries()) {
      if (this.matchesSubscription(event, sub)) {
        ws.send(JSON.stringify(event));
      }
    }
  }

  private matchesSubscription(event: T3000Event, sub: EventSubscription): boolean {
    // Check event type
    if (!sub.eventTypes.has('*') && !sub.eventTypes.has(event.type)) {
      return false;
    }

    // Check device ID
    if (sub.deviceIds.size > 0 && 'deviceId' in event) {
      if (!sub.deviceIds.has(event.deviceId)) {
        return false;
      }
    }

    // Check severity
    if (sub.severity && 'severity' in event) {
      if (event.severity !== sub.severity) {
        return false;
      }
    }

    return true;
  }
}
```

Usage Examples

Subscribe to Alarm Events

TYPESCRIPT

```typescript
import eventBus from './eventBus';

// Listen for all alarms
const unsubscribe = eventBus.on('alarm', (event) => {
  console.log(`Alarm: ${event.message}`);

  if (event.severity === 'critical') {
    sendEmailNotification(event);
  }
});

// Unsubscribe later
unsubscribe();
```

Emit Custom Event

TYPESCRIPT

```typescript
eventBus.emit({
  id: generateEventId(),
  type: 'alarm',
  timestamp: Date.now(),
  source: 'temperature-monitor',
  alarmId: 'ALM_001',
  deviceId: 389001,
  point: 'IN1',
  severity: 'high',
  message: 'Temperature exceeded high limit',
  value: 85.2,
  threshold: 80.0
});
```

Query Event History

```typescript
const logger = new EventLogger('./events.db');

const criticalAlarms = await logger.queryEvents({
  type: 'alarm',
  severity: 'critical',
  startTime: Date.now() - 86400000,  // Last 24 hours
  limit: 50
});
```

Next Steps

- REST API
- WebSocket API
- Modbus Protocol